

# 中国科学院大学

## 《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术  
实验项目编号 2 实验名称 单周期 RISC-V 处理器核心设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

### 一、 逻辑电路结构说明

本次实验基于 RISC-V RV32I 指令集, 完成了单周期处理器核心的模块化设计, 包含寄存器堆、ALU、移位器、CPU 顶层四大核心模块, 各模块的设计细节、代码实现与逻辑说明如下:

1. 寄存器堆设计寄存器堆是处理器的通用寄存器存储单元, 实现 32 个 32 位通用寄存器的异步读、同步写操作, 硬件强制保证 x0 寄存器恒为 0, 不可写入。模块核心逻辑分为读端口组合逻辑与写端口时序逻辑两部分, 完整代码如下:

```
`timescale 10 ns / 1 ns
`define DATA_WIDTH 32
`define ADDR_WIDTH 5
module reg_file(
    input          clk,
    input [`ADDR_WIDTH - 1:0] waddr,
    input [`ADDR_WIDTH - 1:0] raddr1,
    input [`ADDR_WIDTH - 1:0] raddr2,
    input          wen,
    input [`DATA_WIDTH - 1:0] wdata,
    output [`DATA_WIDTH - 1:0] rdata1,
    output [`DATA_WIDTH - 1:0] rdata2
);
// 定义32个32-bit寄存器数组
reg [`DATA_WIDTH - 1:0] rf [0:(1 << `ADDR_WIDTH) - 1];
// 读端口1: 组合逻辑实现$zero恒0
assign rdata1 = ({`DATA_WIDTH{!raddr1}} & rf[raddr1]) | ({`DATA_WIDTH{~raddr1}} &
    `DATA_WIDTH'd0);
// 读端口2: 组合逻辑实现$zero恒0
assign rdata2 = ({`DATA_WIDTH{!raddr2}} & rf[raddr2]) | ({`DATA_WIDTH{~raddr2}} &
    `DATA_WIDTH'd0);
// 写端口: 同步写 + 禁止写$zero
```

```

always @(posedge clk) begin
    if (wen && (waddr != `ADDR_WIDTH'b0)) begin
        rf[waddr] <= wdata;
    end
end
endmodule

```

模块通过位运算实现零寄存器的恒 0 约束，无需额外的分支判断，保证了读操作的组合逻辑延迟最小化；写操作仅在时钟上升沿、写使能有效且写地址非 0 时执行，从硬件层面避免了零寄存器被修改的风险。

2. **ALU 设计**算术逻辑单元 (ALU) 是处理器的运算核心，支持加法、减法、按位与/或/异或/或非、有符号比较、无符号比较共 8 种运算，复用同一套加法器实现加减与比较运算，同时生成溢出、进位、零标志位，完整代码如下：

```

`timescale 10 ns / 1 ns
`define DATA_WIDTH 32
module alu(
    input [`DATA_WIDTH - 1:0] A,
    input [`DATA_WIDTH - 1:0] B,
    input [2:0] ALUop,
    output Overflow,
    output CarryOut,
    output Zero,
    output [`DATA_WIDTH - 1:0] Result
);
// 减法控制信号：SUB/SLT/SLTU时执行补码减法
wire sub_ctrl = (ALUop == 3'b110) || (ALUop == 3'b111) || (ALUop == 3'b011);
// B操作数处理：减法时取反，配合加法器实现A-B = A + (~B) + 1
wire [`DATA_WIDTH - 1:0] B_processed = sub_ctrl ? ~B : B;
// 加法器核心：复用实现ADD/SUB/SLT/SLTU，最高位为进位/借位标志
wire [32:0] add_result = A + B_processed + sub_ctrl;
// 逻辑运算并行计算
wire [`DATA_WIDTH - 1:0] and_result = A & B;
wire [`DATA_WIDTH - 1:0] or_result = A | B;
wire [`DATA_WIDTH - 1:0] xor_result = A ^ B;
wire [`DATA_WIDTH - 1:0] nor_result = ~(A | B);
// SLT有符号比较结果
wire slt_flag = add_result[31] ^ Overflow;
wire [`DATA_WIDTH - 1:0] slt_result;
assign slt_result[0] = slt_flag;
assign slt_result[31:1] = 31'd0;
// SLTU无符号比较结果
wire sltu_flag = ~add_result[32];
wire [`DATA_WIDTH - 1:0] sltu_result;
assign sltu_result[0] = sltu_flag;
assign sltu_result[31:1] = 31'd0;
// 运算结果多路选择
assign Result = (ALUop == 3'b000) ? and_result :
                (ALUop == 3'b001) ? or_result :

```

```

        (ALUop == 3'b100) ? xor_result :
        (ALUop == 3'b101) ? nor_result :
        (ALUop == 3'b010) ? add_result[31:0] :
        (ALUop == 3'b110) ? add_result[31:0] :
        (ALUop == 3'b111) ? slt_result :
        (ALUop == 3'b011) ? sltu_result :
        `DATA_WIDTH'd0;
// 标志位生成
assign Zero = (Result == `DATA_WIDTH'd0);
assign Overflow = ((ALUop == 3'b010) || (ALUop == 3'b110) || (ALUop == 3'b111)) ?
    ((A[31] == B_processed[31]) && (A[31] != add_result[31])) :
    1'b0;
assign CarryOut = (ALUop == 3'b010) ? add_result[32] :
    (ALUop == 3'b110) ? ~add_result[32] :
    (ALUop == 3'b111) ? ~add_result[32] :
    (ALUop == 3'b011) ? ~add_result[32] :
    1'b0;
endmodule

```

模块采用并行运算架构，所有运算逻辑同时执行，仅通过多路选择器输出最终结果，保证了运算的低延迟；通过 `sub_ctrl` 信号复用加法器，大幅减少了硬件资源消耗，同时保证了加减运算与比较运算的结果一致性。

3. 移位器设计移位器模块实现 32 位数据的逻辑左移、逻辑右移、算术右移三种操作，移位位数为 5 位，支持 R 型与 I 型移位指令，完整代码如下：

```

`timescale 10 ns / 1 ns
`define DATA_WIDTH 32
module shifter (
    input [`DATA_WIDTH - 1:0] A,
    input [4:0] B,
    input [1:0] Shiftop,
    output [`DATA_WIDTH - 1:0] Result
);
// 逻辑左移
wire [`DATA_WIDTH - 1:0] sll_result;
assign sll_result = A << B;
// 逻辑右移
wire [`DATA_WIDTH - 1:0] srl_result;
assign srl_result = A >> B;
// 算术右移符号掩码生成
wire [`DATA_WIDTH - 1:0] sign_mask;
wire [`DATA_WIDTH - 1:0] sra_result;
assign sign_mask =
    (B == 5'd0) ? 32'h0000_0000 :
    A[31] ? (32'hffff_ffff << (6'd32 - {1'b0, B})) :
    32'h0000_0000;
assign sra_result = srl_result | sign_mask;
// 最终结果选择
assign Result =
    (Shiftop == 2'b00) ? sll_result :

```

```

        (Shiftop == 2'b10) ? srl_result :
        (Shiftop == 2'b11) ? sra_result :
            32'd0;
endmodule

```

模块针对算术右移做了专门的硬件设计, 通过符号掩码与逻辑右移结果的按位或实现符号位填充, 避免了 Verilog 默认运算符对无符号数的移位限制, 保证了负数算术右移的正确性。

4. 单周期 CPU 顶层设计顶层模块整合所有子模块, 完成 RISC-V RV32I 指令的完整执行流程, 包含指令解析、控制信号生成、数据通路选择、内存访问控制、PC 更新逻辑五大核心部分, 完整代码如下:

```

`timescale 10 ns / 1 ns
module simple_cpu(
    input          clk,
    input          rst,

    output [31:0]  PC,
    input  [31:0]  Instruction,

    output [31:0]  Address,
    output          MemWrite,
    output [31:0]  Write_data,
    output [ 3:0]  Write_strb,

    input  [31:0]  Read_data,
    output          MemRead
);
// ===== 1. 指令字段解析 (RISC-V RV32I) =====
wire [6:0] opcode = Instruction[6:0];
wire [4:0] rd      = Instruction[11:7];
wire [2:0] funct3  = Instruction[14:12];
wire [6:0] funct7  = Instruction[31:25];
wire [4:0] rs1     = Instruction[19:15];
wire [4:0] rs2     = Instruction[24:20];
wire [11:0] imm_I  = Instruction[31:20];
wire [11:0] imm_S  = {Instruction[31:25], Instruction[11:7]};
wire [12:0] imm_B  = {Instruction[31], Instruction[7], Instruction[30:25], Instruction[11:8],
    1'b0};
wire [19:0] imm_U  = Instruction[31:12];
wire [20:0] imm_J  = {Instruction[31], Instruction[19:12], Instruction[20], Instruction[30:21],
    1'b0};
// 立即数符号扩展
wire [31:0] imm_I_sext = {{20{imm_I[11]}}, imm_I};
wire [31:0] imm_S_sext = {{20{imm_S[11]}}, imm_S};
wire [31:0] imm_B_sext = {{19{imm_B[12]}}, imm_B};
wire [31:0] imm_U_sext = {imm_U, 12'd0};
wire [31:0] imm_J_sext = {{11{imm_J[20]}}, imm_J};
// ===== 2. 寄存器堆实例化 =====
wire          RF_wen;
wire [4:0]    RF_waddr;

```

```

wire [31:0] RF_wdata;
wire [31:0] read_data_1;
wire [31:0] read_data_2;
reg_file rf_inst (
    .clk(clk),
    .waddr(RF_waddr),
    .raddr1(rs1),
    .raddr2(rs2),
    .wen(RF_wen),
    .wdata(RF_wdata),
    .rdata1(read_data_1),
    .rdata2(read_data_2)
);
// ===== 3. 控制信号生成 (组合逻辑) =====
wire is_R_type = (opcode == 7'b0110011);
wire is_OP_IMM = (opcode == 7'b0010011);
wire is_LOAD = (opcode == 7'b0000011);
wire is_I_type = is_OP_IMM || is_LOAD;
wire is_S_type = (opcode == 7'b0100011);
wire is_B_type = (opcode == 7'b1100011);
wire is_U_type = (opcode == 7'b0110111) || (opcode == 7'b0010111);
wire is_J_type = (opcode == 7'b1101111) || (opcode == 7'b1100111);
wire RegWrite;
wire ALUSrc;
wire MemtoReg;
wire Branch;
wire Jump;
assign RegWrite = (is_R_type) || (is_I_type) || (is_U_type) || (is_J_type);
assign ALUSrc = is_OP_IMM || is_LOAD || is_S_type || (opcode == 7'b1100111);
assign MemtoReg = (opcode == 7'b0000011);
assign Branch = is_B_type;
assign Jump = is_J_type;
assign RF_wen = RegWrite;
assign RF_waddr = rd;
// ===== 4. ALU与移位器控制信号生成 =====
wire [2:0] alu_op;
wire [1:0] shift_op;
wire is_shift_inst;
// ALU操作码生成
assign alu_op = is_R_type ?
    (funct7 == 7'b0100000 && funct3 == 3'b000) ? 3'b110 :
    (funct3 == 3'b000) ? 3'b010 :
    (funct3 == 3'b010) ? 3'b111 :
    (funct3 == 3'b011) ? 3'b011 :
    (funct3 == 3'b100) ? 3'b100 :
    (funct3 == 3'b110) ? 3'b001 :
    (funct3 == 3'b111) ? 3'b000 :
    3'b010 :
    is_I_type ?

```

```

        (opcode == 7'b0010011) ?
            (funct3 == 3'b000) ? 3'b010 :
            (funct3 == 3'b010) ? 3'b111 :
            (funct3 == 3'b011) ? 3'b011 :
            (funct3 == 3'b100) ? 3'b100 :
            (funct3 == 3'b110) ? 3'b001 :
            (funct3 == 3'b111) ? 3'b000 :
            3'b010 :
        3'b010 :
    is_S_type ? 3'b010 :
    is_B_type ? 3'b110 :
    is_U_type ? 3'b010 :
    is_J_type ? 3'b010 :
    3'b010;
// 移位指令识别与操作码生成
wire is_OP  = (opcode == 7'b0110011);
wire is_SLL = is_OP  && (funct3 == 3'b001) && (funct7 == 7'b0000000);
wire is_SRL = is_OP  && (funct3 == 3'b101) && (funct7 == 7'b0000000);
wire is_SRA = is_OP  && (funct3 == 3'b101) && (funct7 == 7'b0100000);
wire is_SLLI = is_OP_IMM && (funct3 == 3'b001) && (Instruction[31:25] == 7'b0000000);
wire is_SRLI = is_OP_IMM && (funct3 == 3'b101) && (Instruction[31:25] == 7'b0000000);
wire is_SRAI = is_OP_IMM && (funct3 == 3'b101) && (Instruction[31:25] == 7'b0100000);
assign is_shift_inst = is_SLL || is_SRL || is_SRA || is_SLLI || is_SRLI || is_SRAI;
assign shift_op =
    (is_SLL || is_SLLI) ? 2'b00 :
    (is_SRL || is_SRLI) ? 2'b10 :
    (is_SRA || is_SRAI) ? 2'b11 :
    2'b00;
// ===== 5. 操作数输入选择 =====
wire [31:0] alu_input_1 = (opcode == 7'b0010111) ? PC : read_data_1;
wire [31:0] alu_input_2_normal;
assign alu_input_2_normal = ALUSrc ? (
    (is_I_type)                ? imm_I_sext :
    (is_S_type)                ? imm_S_sext :
    (is_J_type && (opcode == 7'b1100111)) ? imm_I_sext :
    read_data_2
) : read_data_2;
// 移位器输入
wire [31:0] shift_input = read_data_1;
wire [4:0] shift_amount = is_R_type ? read_data_2[4:0] : imm_I[4:0];
// ===== 6. ALU与移位器实例化 =====
wire [31:0] alu_result_normal;
wire        zero_flag_alu;
wire        overflow_flag;
wire        carryout_flag;
alu alu_inst (
    .A        (alu_input_1),
    .B        (alu_input_2_normal),
    .ALUOp    (alu_op),

```

```

        .Overflow (overflow_flag),
        .CarryOut (carryout_flag),
        .Zero      (zero_flag_alu),
        .Result     (alu_result_normal)
    );
    wire [31:0] shift_result;
    shifter shift_inst (
        .A      (shift_input),
        .B      (shift_amount),
        .Shiftop(shift_op),
        .Result  (shift_result)
    );
    // 最终运算结果选择
    wire [31:0] alu_result = is_shift_inst ? shift_result : alu_result_normal;
    wire        zero_flag = is_shift_inst ? (shift_result == 32'd0) : zero_flag_alu;
    // ===== 7. 寄存器写回数据选择 =====
    wire [1:0] mem_addr_low = alu_result[1:0];
    wire [7:0] load_byte =
        (mem_addr_low == 2'b00) ? Read_data[7:0] :
        (mem_addr_low == 2'b01) ? Read_data[15:8] :
        (mem_addr_low == 2'b10) ? Read_data[23:16] :
                                Read_data[31:24];
    wire [15:0] load_half =
        mem_addr_low[1] ? Read_data[31:16] : Read_data[15:0];
    wire [31:0] load_data =
        (funct3 == 3'b000) ? {{24{load_byte[7]}}, load_byte} :
        (funct3 == 3'b001) ? {{16{load_half[15]}}, load_half} :
        (funct3 == 3'b010) ? Read_data :
        (funct3 == 3'b100) ? {24'b0, load_byte} :
        (funct3 == 3'b101) ? {16'b0, load_half} :
                                Read_data;
    assign RF_wdata =
        RegWrite ? (
            MemtoReg ? load_data :
            (opcode == 7'b0110111) ? imm_U_sext :
            (opcode == 7'b0010111) ? PC + imm_U_sext :
            ((opcode == 7'b1101111) || (opcode == 7'b1100111)) ? (PC + 32'd4) :
            alu_result
        ) : 32'h00000000;
    // ===== 8. 内存访问控制 =====
    wire is_memory_access = is_S_type || (opcode == 7'b0000011);
    assign MemWrite = is_S_type;
    assign MemRead = (opcode == 7'b0000011);
    assign Address = is_memory_access ? {alu_result[31:2], 2'b00} : 32'd0;
    // 存储指令字节使能与写入数据处理
    wire is_SB = is_S_type && (funct3 == 3'b000);
    wire is_SH = is_S_type && (funct3 == 3'b001);
    wire is_SW = is_S_type && (funct3 == 3'b010);
    wire [4:0] mem_shift = {mem_addr_low, 3'b000};

```

```

assign Write_data =
    is_SB ? ({24'b0, read_data_2[7:0]} << mem_shift) :
    is_SH ? ({16'b0, read_data_2[15:0]} << mem_shift) :
    is_SW ? read_data_2 :
        32'd0;
assign Write_strb =
    is_SB ? (4'b0001 << mem_addr_low) :
    is_SH ? (4'b0011 << mem_addr_low) :
    is_SW ? 4'b1111 :
        4'b0000;
// ===== 9. PC更新逻辑 =====
reg [31:0] PC_reg;
assign PC = PC_reg;
wire [31:0] PC_next;
always @(posedge clk or posedge rst) begin
    if (rst) begin
        PC_reg <= 32'h00000000;
    end
    else begin
        PC_reg <= PC_next;
    end
end
end
wire      branch_taken;
// ===== Branch 标志位判断 =====
wire branch_eq = zero_flag_alu;
wire branch_ne = ~zero_flag_alu;
wire branch_lt_signed = alu_result_normal[31] ^ overflow_flag;
wire branch_ge_signed = ~branch_lt_signed;
wire branch_lt_unsigned = carryout_flag;
wire branch_ge_unsigned = ~carryout_flag;
assign branch_taken = Branch && (
    ((funct3 == 3'b000) && branch_eq)    || // BEQ
    ((funct3 == 3'b001) && branch_ne)    || // BNE
    ((funct3 == 3'b100) && branch_lt_signed) || // BLT
    ((funct3 == 3'b101) && branch_ge_signed) || // BGE
    ((funct3 == 3'b110) && branch_lt_unsigned) || // BLTU
    ((funct3 == 3'b111) && branch_ge_unsigned) // BGEU
);
assign PC_next =
    (Jump && (opcode == 7'b1101111)) ? PC + imm_J_sext :
    (Jump && (opcode == 7'b1100111)) ? {alu_result[31:1], 1'b0} :
    (Branch && branch_taken)      ? PC + imm_B_sext :
    PC + 32'd4;
endmodule

```

模块的核心设计思路与运行流程如下：

整体设计思路：采用自底向上的模块化设计与单周期执行架构，将处理器划分为指令解析单元、控制信号生成单元、数据通路（寄存器堆、ALU、移位器）、内存访问接口、PC 更新逻辑五大核心部分。所有指令的取指、



译码、执行、访存、写回操作均在单个时钟周期内通过组合逻辑完成,实现 CPI=1 的执行效率,同时通过清晰的模块划分保证逻辑的可维护性与可扩展性。

指令的完整执行流程:

- (a) 取指与指令解析: 处理器根据当前 PC 值从指令存储器取出 32 位指令, 首先对指令进行字段拆分, 提取 **opcode**、**rd**、**funct3**、**funct7**、**rs1**、**rs2** 等核心字段; 同时根据指令类型 (**I/S/B/U/J**) 提取对应的立即数字段, 并进行符号扩展, 为后续运算准备好立即数操作数。
- (b) 控制信号生成: 控制单元根据 **opcode** 识别指令的大类 (**R/I/S/B/U/J**), 再结合 **funct3**、**funct7** 字段进一步细化指令类型, 生成一系列控制信号: 包括寄存器写使能 **RegWrite**、ALU 第二操作数选择信号 **ALUSrc**、内存写回选择信号 **MemtoReg**、分支使能 **Branch**、跳转使能 **Jump** 等, 为数据通路的多路选择器提供选择依据。
- (c) 寄存器读与操作数选择: 根据 **rs1**、**rs2** 地址从寄存器堆异步读取两个源操作数; 同时根据 **ALUSrc** 控制信号, 为 ALU 的第二操作数选择数据源 (寄存器数据或符号扩展后的立即数), 为 **AUIPC** 指令特殊选择 PC 作为 ALU 的第一操作数。
- (d) 执行与移位操作: ALU 根据 **alu\_op** 控制信号完成加法、减法、按位运算、比较运算等核心算术逻辑操作; 移位器则根据 **shift\_op** 控制信号独立处理逻辑左移、逻辑右移、算术右移操作; 最终通过多路选择器, 根据是否为移位指令选择 ALU 结果或移位器结果作为本周期的运算输出。
- (e) 内存访问与数据写回准备: 对于 **Load** 指令, 根据 ALU 计算出的内存地址, 从数据存储器读取数据, 并根据 **funct3** 字段和地址低两位的偏移量, 完成字节/半字的提取与符号扩展; 对于 **Store** 指令, 将寄存器数据根据偏移量进行移位, 并生成对应的字节使能信号 **Write\_strb**, 为内存写入做好准备; 同时根据指令类型, 选择 ALU 结果、内存数据、PC+4、立即数等作为最终的寄存器写回数据。
- (f) PC 更新与指令提交: 在时钟上升沿, 根据分支条件判断 (**Branch** 与比较结果) 和跳转指令 (**Jump**), 计算下一条指令的 PC 地址: 顺序执行时 PC+4, 分支跳转时 PC+ 偏移量, **JALR** 跳转时 ALU 结果最低位置 0; 同时寄存器堆在时钟上升沿完成写回操作, 一条指令执行完毕, 处理器进入下一个时钟周期。

模块通过组合逻辑完成所有指令的译码与执行, 所有操作在单个时钟周期内完成, 实现了 CPI=1 的单周期处理器架构。整体数据通路结构如 Fig. 1 所示。

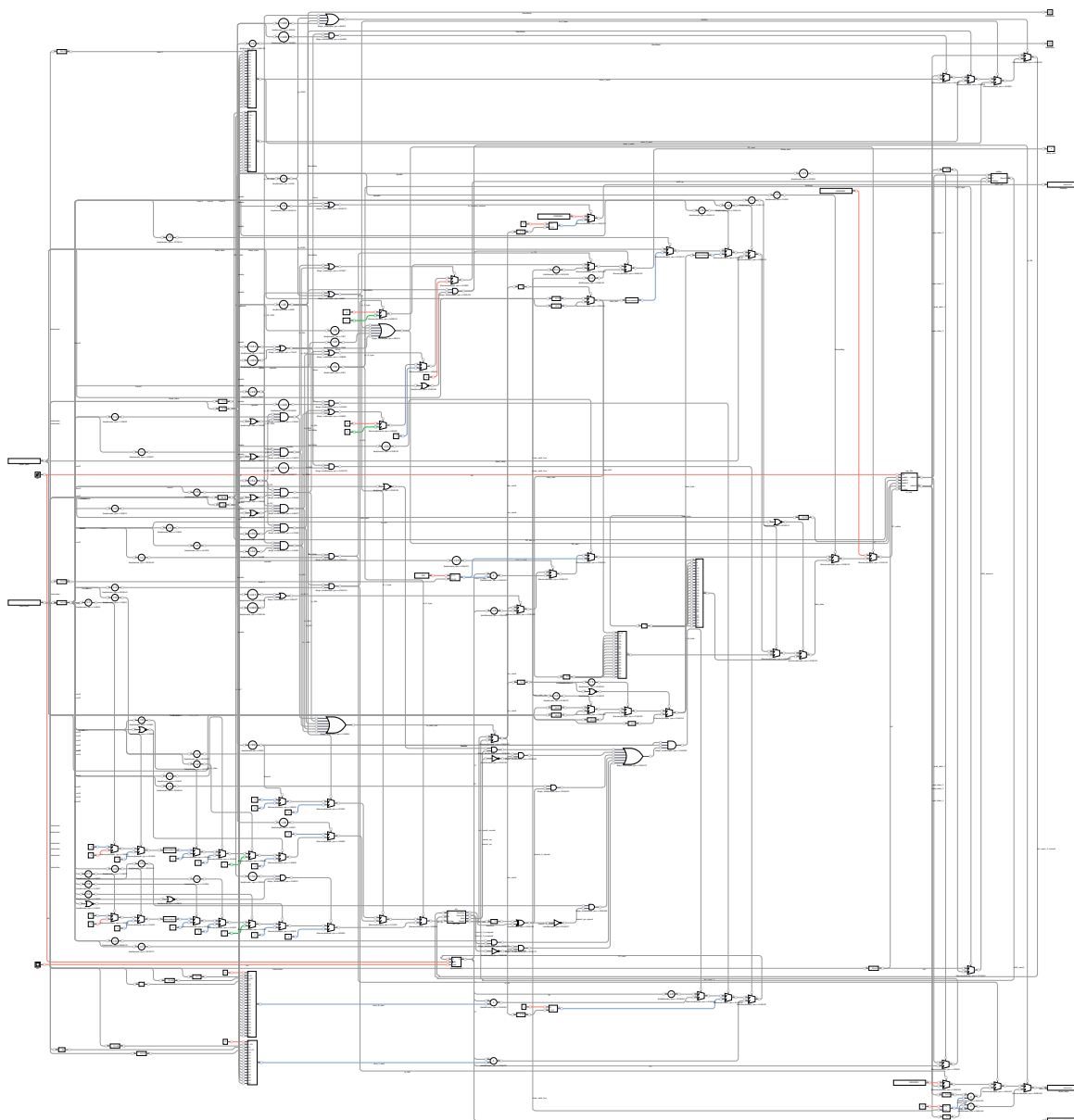


图 1: 单周期 RISC-V 处理器核心整体结构框图

## 二、 实验过程中遇到的问题、对问题的思考过程及解决方法

本次实验的核心难点在于处理器逻辑的严谨性与边界情况的处理, 实验过程中遇到的核心问题、思考与解决方法如下:

### 1. 移位器算术右移的符号位污染问题

问题现象:

最初通过嵌套三目运算符直接使用 Verilog 移位运算符实现算术右移, 仿真中发现负数算术右移时高位未补 1, 移位器功能完全失效。

思考过程:

通过仿真波形定位到, Verilog 中移位运算符的行为由操作数的符号类型决定, 代码中定义的操作数为无符号 wire 类型, 即使使用算术右移运算符 `>>>`, 也会被当作逻辑右移处理, 高位统一补 0, 导致有符号数的移

位结果被污染。

解决方法：

放弃直接使用运算符实现算术右移，改用纯组合逻辑生成符号掩码的方案。先对操作数做逻辑右移，再根据操作数的符号位和移位位数，生成高位补 1 的掩码，将掩码与逻辑右移结果按位或，正确实现了算术右移的符号位填充，彻底解决了符号位污染问题。

## 2. 寄存器写使能的条件判断错误问题

问题现象：

仿真中 R 型、I 型指令的执行结果无法正确写入寄存器堆，多数指令的写回功能异常。

思考过程：

通过波形调试发现，顶层模块的 RF\_wen 信号被错误拉低。最初设计时，在顶层 RegWrite 信号的逻辑中额外添加了 `rd != 5'b0` 的限制条件，而 reg\_file 模块内部已经实现了“写地址为 0 时禁止写入”的硬件保护，两层冗余判断导致仿真场景下写使能逻辑不一致，导致仿真不通过。

解决方法：

删除顶层 RegWrite 信号中对 rd 地址的额外判断，仅保留指令类型的判断逻辑，完全复用 reg\_file 模块内部的零寄存器写保护功能，既简化了控制逻辑，又消除了冗余判断导致的写使能异常问题。

## 3. 内存读写地址对齐与偏移处理错误问题

问题现象：

LB、LH、SB、SH 等字节/半字访存指令执行结果与预期不符，无法正确实现非对齐的字节级访存操作。

思考过程：

最初误以为 ALU 计算出的有效地址就是直接的内存字地址，忽略了 RISC-V 架构中 32 位内存地址的低两位是字内字节偏移量，导致访存时无法正确定位到字内的对应字节/半字；同时写入数据的移位逻辑和字节使能信号 Write\_strb 的生成逻辑混乱，没有根据偏移量做正确的位宽对齐。

解决方法：

首先修正内存地址生成逻辑，将 ALU 结果的高 30 位作为字地址，低两位强制清零，保证内存访问的字对齐；然后拆分字节/半字的读写逻辑，根据地址低两位的偏移量，分别实现 Load 指令的字节/半字提取与符号扩展，以及 Store 指令的写入数据移位和字节使能生成，最终实现了符合 RISC-V 规范的字节/半字访存功能。

## 4. 嵌套三目运算符的语法与逻辑错误问题

问题现象：

代码编译阶段出现语法报错，部分组合逻辑信号在仿真中出现不定态 X，导致功能异常。

思考过程：

最初通过多层嵌套三目运算符实现多条件的多路选择器逻辑时，遗漏了最外层条件为假时的默认分支，导致 Verilog 语法不完整，综合器无法正确推断组合逻辑，不仅触发编译报错，还会生成不必要的锁存器，导致仿真中出现不定态。

解决方法：

梳理所有嵌套三目运算符的逻辑，为每一层条件判断都补充完整的真分支和假分支，尤其是最外层的默认值，保证所有组合逻辑信号都有明确的驱动，彻底消除了语法错误和不定态问题，同时避免了组合逻辑中锁存器的生成。

## 5. 实验心得与扩展想法

本次实验通过自底向上的模块化设计,完成了单周期 RISC-V 处理器核心的实现,从基础的寄存器堆、ALU 模块,到完整的指令译码与数据通路,每一个细节都需要严谨的逻辑设计,让我对处理器的底层执行原理有了从理论到实践的深度理解,也深刻体会到了计算机组成原理中“模块化设计、边界情况处理”的核心思想。特别感谢课程老师的悉心指导,以及同学在实验过程中提供的思路帮助,让我能够快速定位并解决多个逻辑 bug。

关于本次实验内容的扩展,初步技术路线如下:

第一,将单周期处理器改造为经典 5 级流水线处理器,增加取指、译码、执行、访存、写回五级流水线寄存器,解决数据冒险和控制冒险问题,大幅提升处理器的运行频率;

第二,扩展支持更多 RV32I 指令集的指令,包括 CSR 系统控制指令、环境调用指令,完善处理器的功能完整性;

第三,添加指令 Cache 与数据 Cache 模块,采用直接映射或组相连映射架构,提升处理器的访存效率;

第四,实现简易的异常与中断处理机制,支持异常的捕获、处理与返回,完善处理器的可靠性与实用性。

## 三、对一些思考题的理解和回答

### 1. 思考题 1: 为什么单周期 RISC-V 处理器的 CPI(每条指令的时钟周期数)恒为 1? 这种架构有什么优缺点?

单周期处理器的核心设计是,所有指令的取指、译码、执行、访存、写回全流程操作,都在单个时钟周期内完成,每个时钟周期只会执行并完成一条指令,因此每条指令的 CPI 恒为 1。

该架构的优点主要有两点:

一是控制逻辑设计简单,无需流水线寄存器和复杂的冒险处理逻辑,易于理解和实现,非常适合处理器入门学习;

二是指令执行流程无流水线开销,无需处理分支延迟槽等问题,编程模型简洁。

该架构的缺点也十分明显:

一是时钟周期由处理器中最慢的指令(通常是 Load 指令)决定,即使是简单的加法指令也需要等待完整的最长延迟,导致处理器的整体运行频率受限,性能上限较低;

二是硬件资源利用率低,每个时钟周期内,访存单元、ALU 等硬件仅在对应阶段工作,其余时间处于空闲状态,无法实现指令级并行。

### 2. 思考题 2: RISC-V 指令集的立即数格式为什么要分为 I、S、B、U、J 五种类型? 这样设计有什么核心优势?

RISC-V 的立即数格式设计,核心目标是简化硬件实现、降低译码逻辑的复杂度,同时保证指令编码的规整性。五种立即数格式分别对应不同的指令应用场景: I 型用于立即数运算与 Load 指令, S 型用于 Store 指令, B 型用于分支指令, U 型用于长立即数加载, J 型用于跳转指令。

该设计的核心优势主要有三点:

一是所有立即数的符号位都固定在指令的最高位(bit31),硬件仅需要对单个 bit 位做符号扩展,大幅减少了译码阶段的硬件开销;

二是每种格式的立即数字段都尽量固定在指令的相同位置,比如 rs1、rs2、rd 字段的位置在所有指令格式中完全固定,硬件无需额外的多路选择器切换字段位置,提升了译码效率;

三是指令长度固定为 32 位,所有立即数都在指令内编码,无需额外的指令周期读取立即数,保证了单周期指令的执行效率。

3. 思考题 3: 算术右移和逻辑右移的核心区别是什么? 为什么本实验中算术右移需要单独用掩码实现, 而不能直接用 Verilog 的右移运算符?

算术右移和逻辑右移的核心区别在于高位的填充规则: 逻辑右移无论操作数的符号是正还是负, 高位统一填充 0, 适用于无符号数的移位操作; 算术右移高位填充操作数的符号位, 正数补 0、负数补 1, 适用于有符号数的除法等算术运算场景。

本实验中不能直接使用 Verilog 右移运算符实现算术右移, 核心原因是代码中定义的移位操作数为 `wire [31:0]` 类型, 属于无符号数类型。在 Verilog 语法中, 右移运算符的行为由操作数的符号类型决定, 即使使用算术右移运算符 `>>`, 对于无符号数也会按照逻辑右移处理, 高位统一补 0, 无法实现负数的符号位填充。因此本实验需要手动生成符号掩码, 通过逻辑右移结果与掩码的按位或, 正确实现算术右移的功能。

## 四、 实验所耗时间

在课后, 我花费了大约 5 小时完成此次实验。